WinAgent — High-Level System Architecture

Windows 11 AI desktop-control agent · Version 1.0 · Prepared 28 Oct 2026 from the current source tree

WinAgent lets a user describe a computer task in plain language. A planner turns it into a structured, whitelisted plan with measurable success criteria; a policy engine decides what may run automatically and what needs explicit approval; a local Windows agent executes the authorised operations; and the server verifies the real outcome with evidence before marking a task complete.

1. System context

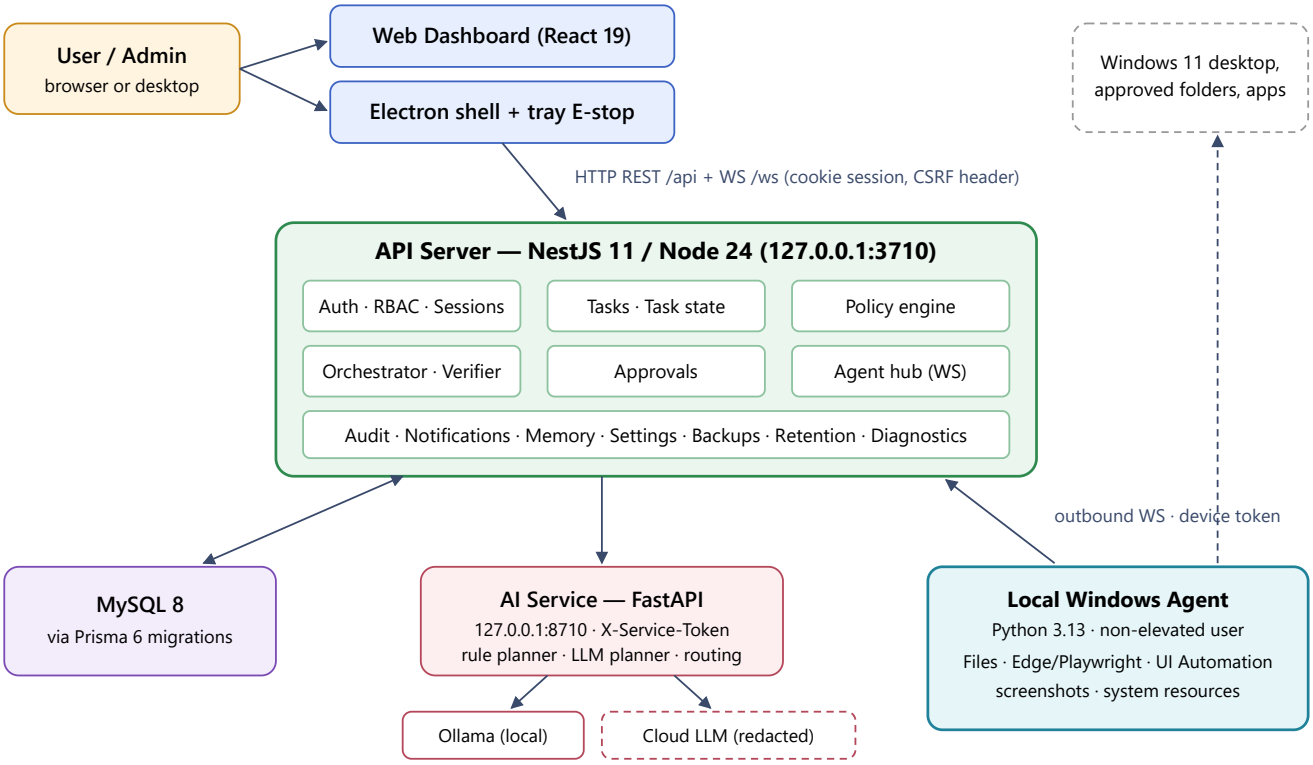


Figure 1 — Components, trust boundaries and connections. All services bind to localhost; the agent only ever connects outward.

2. Components

Component	Technology	Responsibility
Web dashboard `web/`	React 19, TypeScript, Vite	13 pages: Dashboard, New task, Task detail, Live monitor, Approvals, History, AI & models, Policies, Memory, Agent devices, Diagnostics, Security, Settings. Live updates over WebSocket; emergency-stop button on every page; voice input.
Desktop shell `electron/`	Electron 37	Loads the same dashboard (one UI). Sandboxed renderer, context isolation, navigation locked to the dashboard origin, notifications-only IPC, tray emergency stop that writes the agent's local stop file.
API server `server/`	Node 24, NestJS 11, Prisma 6, zod	System of record and control plane: authentication and RBAC, task intake, planning requests, policy evaluation, approvals, sequential orchestration, agent hub, evidence-based verification, audit log, notifications, backups and retention. Serves the built dashboard in production.
AI service `ai-service/`	Python 3.13, FastAPI, httpx	Turns an instruction into a whitelisted plan. Deterministic rule planner first; otherwise model routing (privacy level, complexity, cost limit, resource pressure, connectivity) to Ollama, OpenAI-compatible or Anthropic providers, with fallback. Redacts secrets; stores provider API keys with Windows DPAPI.
Windows agent `agent/`	Python 3.13, websockets, Playwright, pywinauto, psutil	Executes authorised operations on the user's PC inside locally configured approved folders and allow-listed apps. Idempotency journal, local approval checks, emergency stop. Described in the separate Agent Architecture document.
Database	MySQL 8.0	Users, roles, sessions, tasks, steps, executions, evidence, approvals, devices, policies, models, memory, notifications, audit events, config backups.
E2E tests `e2e/`	Playwright	Starts the real stack and real agent and runs the acceptance task (create, read back and verify `hello.txt`).

3. Task lifecycle (end-to-end flow)

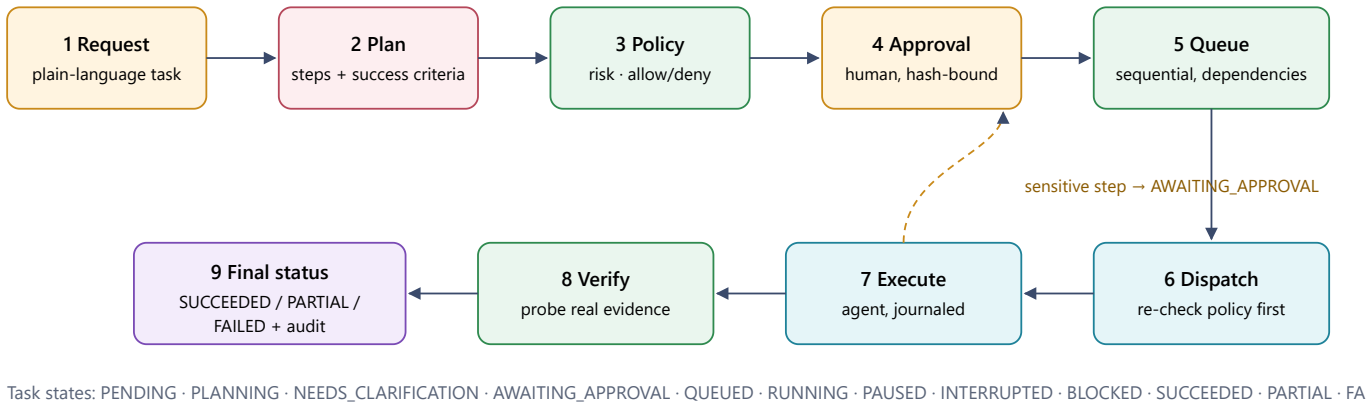


Figure 2 — From request to verified outcome.

- **Planning.** The server sends the instruction, the device's approved roots and apps, the operation catalogue and the configured models to the AI service. The rule planner handles common requests (files, folders, websites, web search, apps, windows, screenshots, resources) without any model call; anything else is routed to a model. Plans are validated against the whitelisted operation catalogue.
- **Policy.** Each step gets a risk level (LOW → CRITICAL) from its operation's base risk plus context (e.g. overwrite raises risk). Folder, website, application and action rules combine with *DENY* > *REQUIRE_APPROVAL* > *ALLOW*. Policy is evaluated at planning time *and again immediately before dispatch*, so a revoked permission also stops queued and running tasks.
- **Approval.** An approval is bound to a hash of the exact action (operation + parameters). If the step changes, the approval no longer matches. The agent independently refuses sensitive operations without an approval hash.
- **Orchestration.** One task runs at a time, ordered by queue position, honouring dependencies (a failed dependency blocks the dependant). Retries apply only to retryable error codes; uncertain operations are checked against the agent's journal and never blindly repeated. A CAPTCHA/OTP/MFA prompt blocks the task for a human.
- **Verification.** Success criteria (`file_exists`, `file_content_equals`, `file_contains`, `step_succeeded`, `page_contains_text`, `window_exists`) are re-checked by sending fresh read-only probes to the agent. Evidence is stored, including SHA-256 hashes for file content.
- **Restart safety.** Tasks that were planning, queued or running when the server stopped become INTERRUPTED and require an explicit Resume.

4. Server module map

Module	Role
auth	First-admin bootstrap, login, cookie sessions, CSRF header check, RBAC guard (roles → permissions), rate limiting.
tasks / task-state	Task intake, plan storage, queue positions, dependencies; a single state machine for all status transitions.
ai (ai-client)	Authenticated client to the AI service for planning and routing previews.
operations	The whitelisted operation catalogue: parameter schemas, base risk, timeouts, URL/host validation.
policy	Pure step evaluation plus persisted rules (FOLDER, WEBSITE, APPLICATION, ACTION).
approvals	Approval requests and decisions, action-hash binding.
orchestrator / verifier	Tick loop, sequential execution, retries, recovery after restart, evidence-based verification.
realtime	Agent hub (device WebSocket, hello/heartbeat, dispatch, cancel, status queries, emergency stop broadcast); dashboard event gateway.
agents	Pairing codes, device registration, revocation.
memory, settings, notifications, audit, backups, system	Workflow skills and memory records, user preferences, in-app notifications, append-only audit trail, config backups, retention, diagnostics, emergency stop / queue control.

5. Data model (grouped)

Area	Entities
Identity & access	User, Role, Permission, RolePermission, UserRole, UserSession
Work	Task, TaskStep, TaskDependency, TaskCheckpoint, TaskExecution, VerificationEvidence
Approval	ApprovalRequest, ApprovalDecision
Devices	AgentDevice, AgentSession, PairingCode
AI & policy	ModelConfig, PermissionPolicy
User data	UserPreference, WorkflowSkill, MemoryRecord, Notification
System	SystemState (emergency stop, queue enabled), AuditEvent, ConfigBackup

Enums: TaskStatus (14 states), StepStatus, ExecutionStatus, EvidenceOutcome, ApprovalStatus, DeviceStatus, RiskLevel (LOW, MEDIUM, HIGH, CRITICAL), PrivacyLevel (LOCAL_ONLY, CLOUD_REDACTED, CLOUD_ALLOWED), PolicyKind, PolicyEffect.

6. Security architecture

Concern	Control
Network exposure	Server, AI service and dashboard bind to 127.0.0.1. Agent connects outbound only; it opens no listening port.
User authentication	Session cookies, CSRF header on state-changing requests, rate limiting, RBAC permissions per endpoint.
Service-to-service	Server → AI service uses a random shared service token generated by `npm run setup:env`.
Device authentication	One-time pairing code → device token (bearer on the WebSocket), stored on the PC with Windows DPAPI; revocable from the dashboard.
Least privilege	Approved folders and allowed apps are configured locally on the agent; the server can narrow but never widen them. Agent refuses to run elevated.
Dangerous actions	Risk scoring, human approval bound to an action hash, double enforcement on agent, backups before overwrite/move/delete.
AI safety & privacy	Whitelisted operations only; secret redaction always; cloud models receive redacted text and are forbidden under LOCAL_ONLY; prompt-injection text detected in web pages.
Secrets at rest	Provider API keys in DPAPI-protected store; `.env` files git-ignored; logs and stored results redacted.
Emergency stop	Dashboard button, Electron tray and agent CLI. Cancels in-flight operations, pauses active tasks, refuses new ones until an admin clears it. Nothing resumes automatically.
Accountability	Audit events for every security-relevant action and every verification.

7. Deployment view

Process	Address	Started by
API server (+ built dashboard)	127.0.0.1:3710	`node server\dist\main.js` (dev: `npm run dev:server`)
Dashboard dev server	127.0.0.1:5173 (proxies /api, /ws)	`npm run dev:web`
AI service	127.0.0.1:8710	`python -m app.main`
Windows agent	outbound to ws://127.0.0.1:3710	`python -m winagent run`
MySQL	localhost:3306	Windows service
Ollama (optional)	127.0.0.1:11434	Ollama service

Single-machine deployment on Windows 11 under a normal user account. Schema changes are applied through version-controlled Prisma migrations (`npm run db:migrate`).

8. Testing strategy

- Server: Jest unit tests (policy, operations, verifier, common) and integration tests against a real MySQL test database and the real AI service, with a simulated agent.
- Web: component tests. AI service and agent: pytest (agent tests use real files, real headless Edge and real UI Automation).
- End-to-end: Playwright acceptance test with the real agent on a dedicated database.

Some PRD features are partial or not yet implemented; the README lists them. See the README for the current Electron shell status.